

Design Pattern III

Recap

Behavioral Design Patterns

how different objects in a system can communicate effectively without being tightly coupled?

Behavioral Design Patterns

- **Definition:** Patterns that identify common communication patterns between objects and realize these patterns.
- **The Goal:** Increase flexibility in carrying out responsibilities.
- Behavioral design patterns are a category of [design patterns](#) that focus on the interactions and communication between objects. They help define how objects collaborate and distribute responsibility among them, making it easier to manage complex control flow and communication in a system.

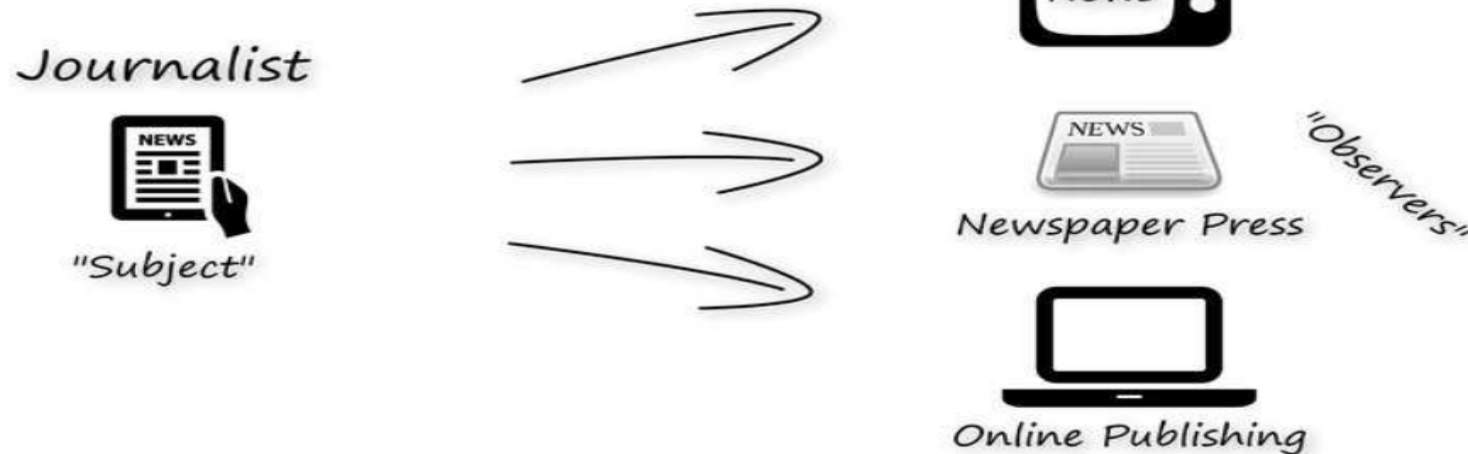
Observer Pattern

"Don't Call Us, We'll Call You."

Observer pattern

Problem??

- An object needs to update others when its state changes, but you don't want them tightly coupled (i.e., you don't want the "Subject" to know exactly who its "Observers" are).

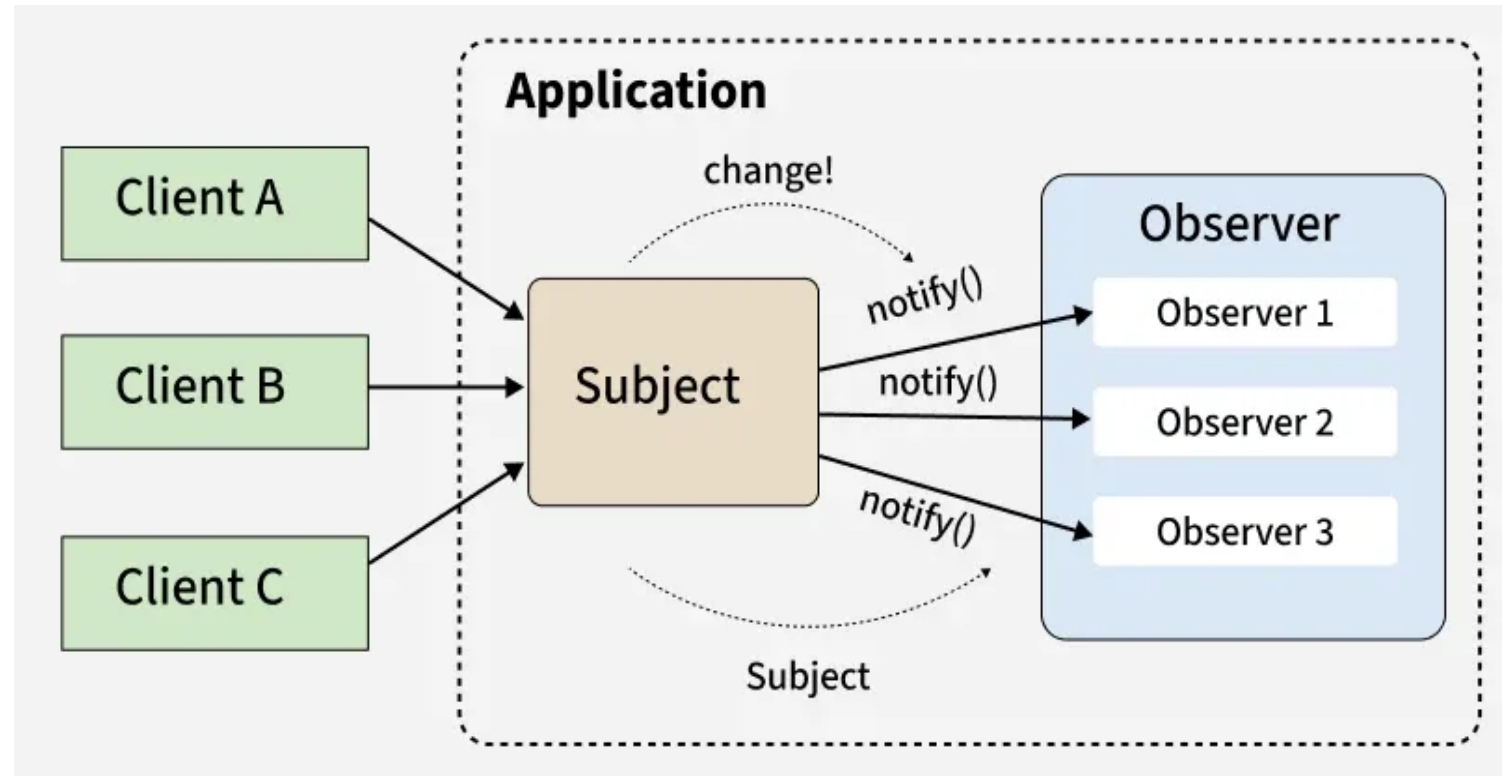


The Solution

- The Subject maintains a list of subscribers. When something happens, it loops through that list and calls a predefined **update()** method on each.

Observer Pattern

Observer is a behavioral design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.



Components of Strategy Pattern

Subject:

- Maintains a list of observers.
- provides methods to add/remove them,
- notifies them of state changes.

Observer:

- Defines an interface with an update() method to ensure all observers receive updates consistently.

Concrete Subject:

- A specific subject that holds actual data

Concrete Observer:

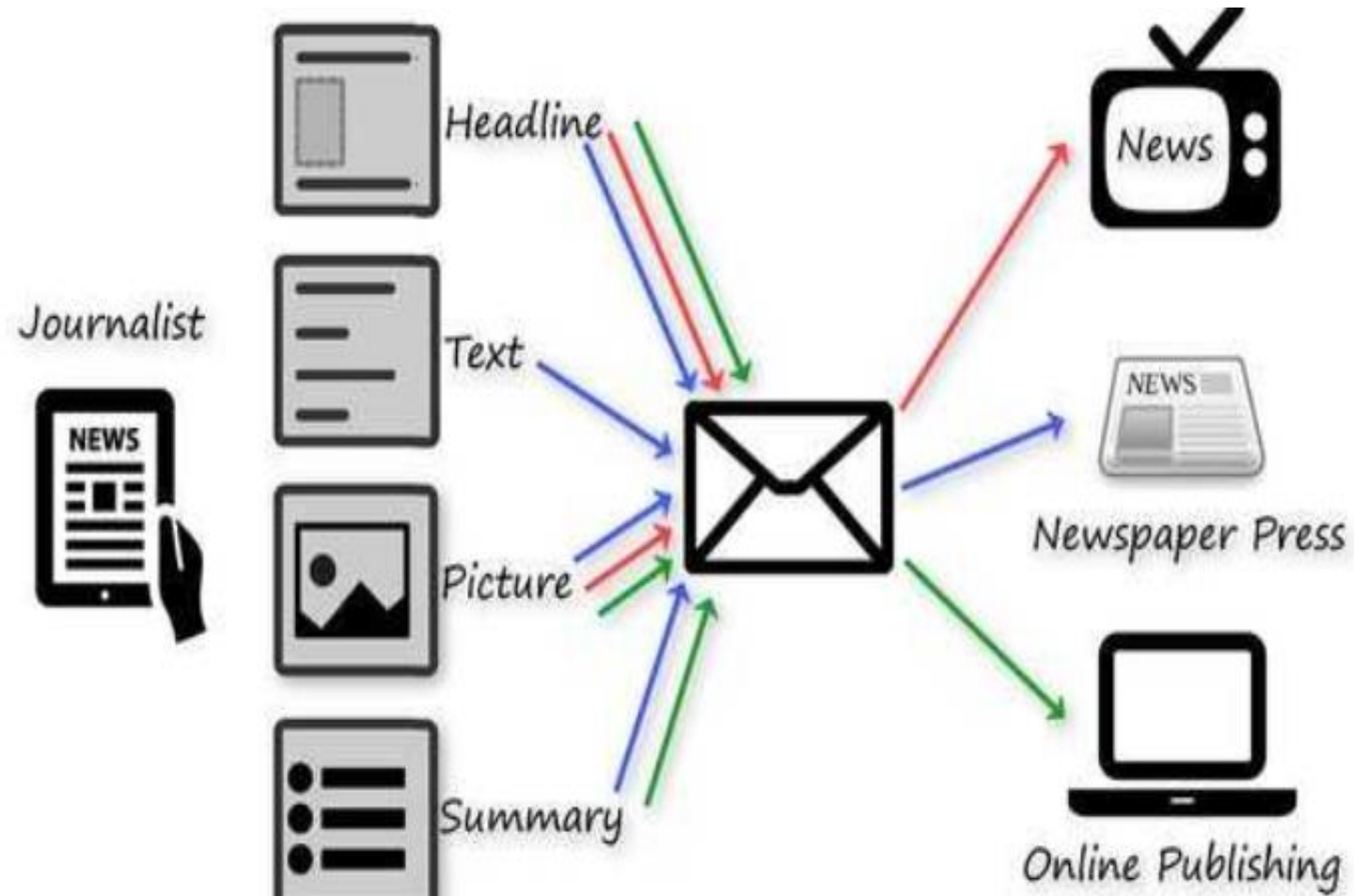
- Implements the observer interface and reacts to subject updates

Take a look

what if we added it
while production

O/C

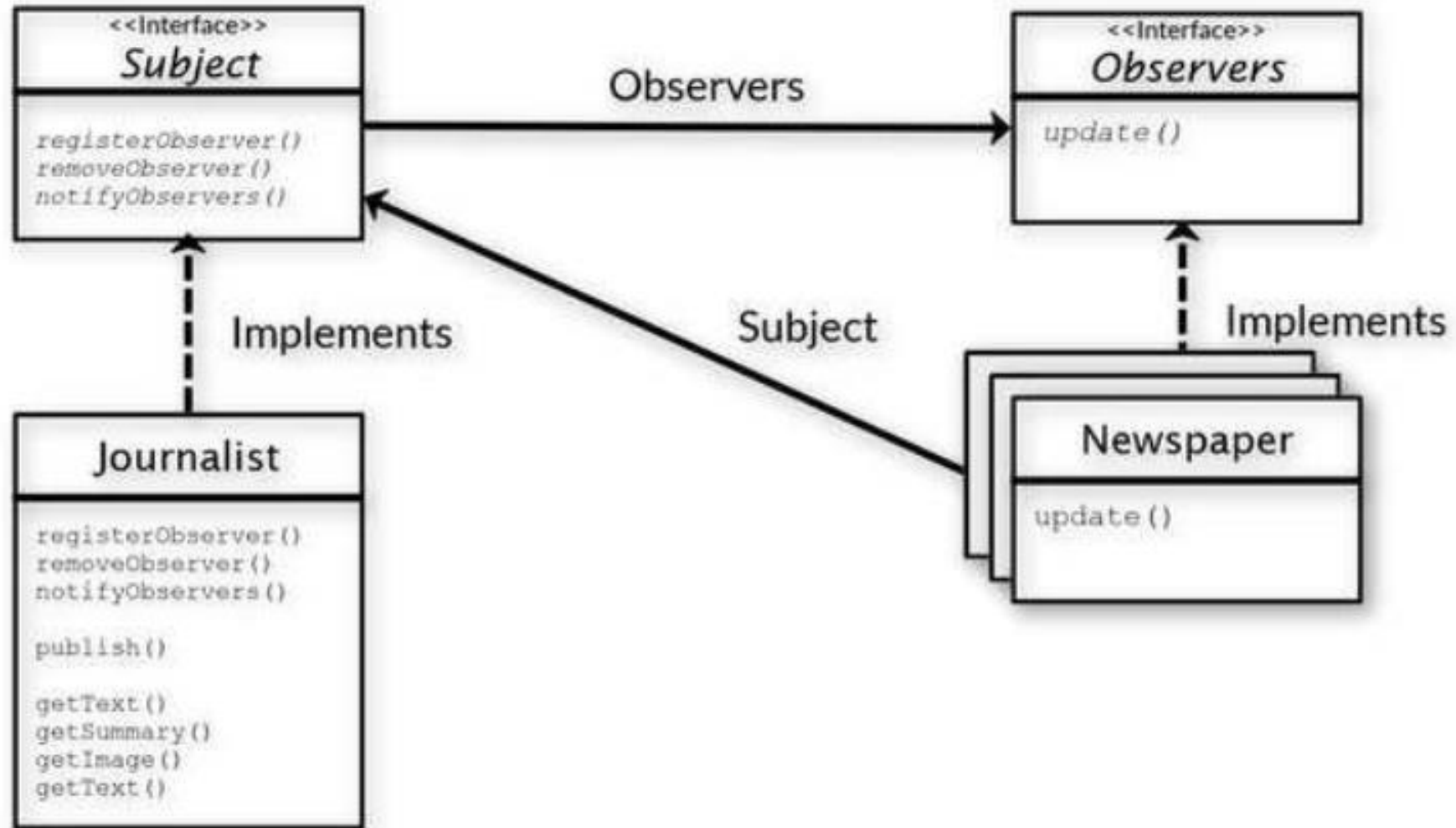
```
class Journalist {  
    TelevisionNews televisionNews;  
    Newspaper newspaper;  
    OnlinePublisher onlinePublisher;  
    public Journalist() {  
        televisionNews = new TelevisionNews();  
        newspaper = new Newspaper();  
        onlinePublisher = new OnlinePublisher();  
    }  
    void publish(String title, String summary, String image, String  
        televisionNews.publish(headline, picture);  
        newspaper.publish(headline, text, picture, summary);  
        onlinePublisher.publish(headline, picture, summary);  
    }  
}
```



1. Not possible to add or remove observers at runtime

2. Too many responsibilities for the Journalist

SOLUTION



```
interface Observer {  
    void update(Subject subject);  
}
```

```
interface Subject {  
    void registerObserver(Observer observer);  
    void removeObserver(Observer observer);  
    void notifyObserver();  
}
```

```
class Newspaper implements Observer {  
    @Override  
    public void update(Subject subject) {  
        if (subject instanceof Journalist) {  
            String title = ((Journalist) subject).getTitle();  
            String summary = ((Journalist) subject).getSummary();  
            String image = ((Journalist) subject).getImage();  
            String text = ((Journalist) subject).getText();  
        }  
    }  
}
```

```
class Journalist implements Subject {  
    private ArrayList observers = new ArrayList<>();  
    private String title, summary, image, text;  
    @Override  
    public void registerObserver(Observer observer) {  
        observers.add(observer);  
    }  
    @Override  
    public void removeObserver(Observer observer) {  
        observers.remove(observer);  
    }  
    @Override  
    public void notifyObserver() {  
        observers.forEach(observer -> observer.update(this));  
    }  
    public void publish(String title, String summary, String image, String text) {  
        this.title = title;  
        this.summary = summary;  
        this.image = image;  
        this.text = text;  
        notifyObserver();  
    }  
}
```

```
public static void main(String [] args) {  
    Journalist journalist = new Journalist();  
    journalist.registerObserver(new Newspaper());  
    journalist.registerObserver(new TVNews());  
    journalist.registerObserver(new OnlinePublishing());  
    journalist.publish("Birds going extinct",  
        "Summary: Research from the university of ...",  
        "Image of Birds",  
        "The number of birds are going down ...");  
}
```

Example 1

Youtube channel

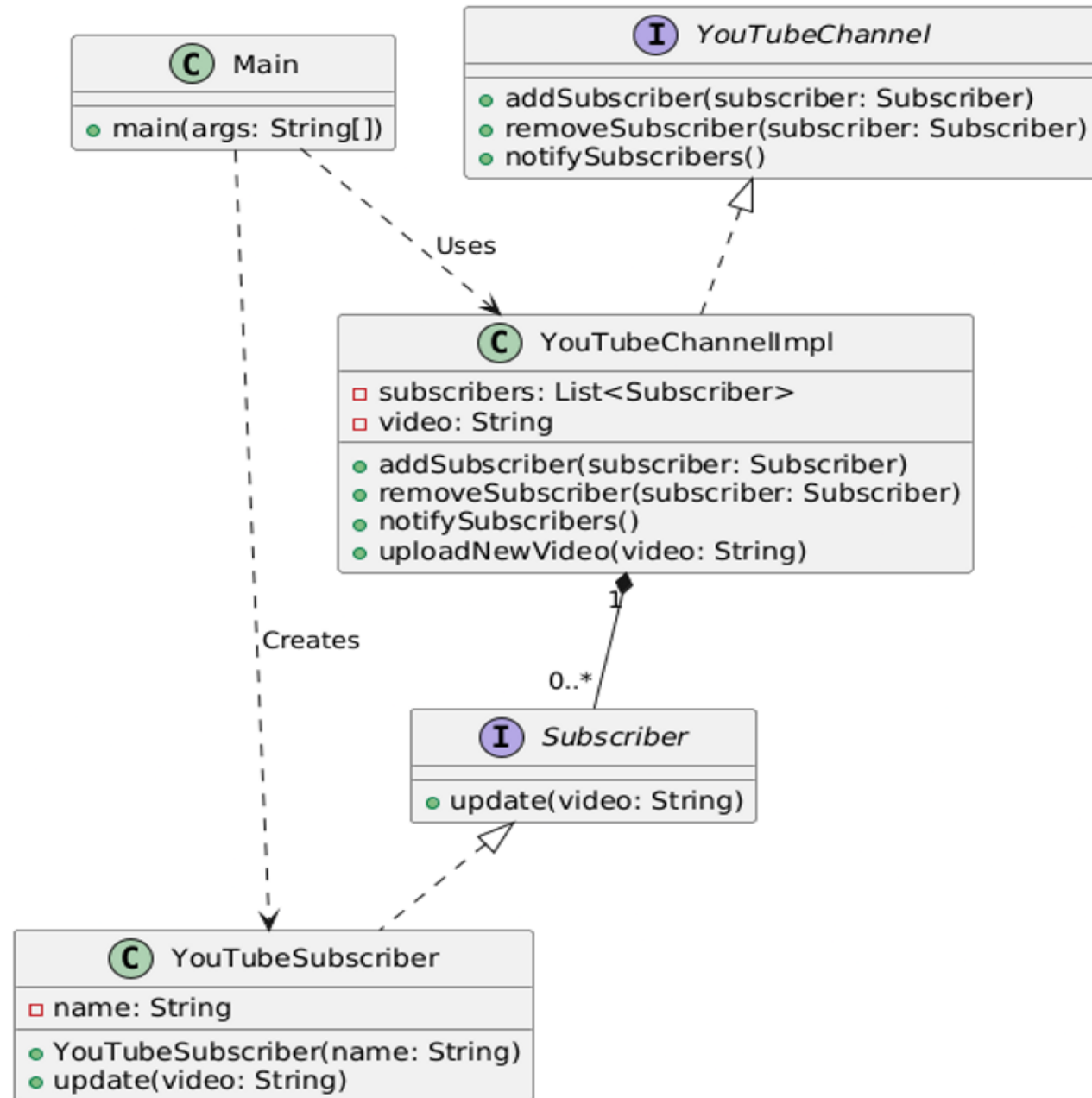
```
1 import java.util.ArrayList;
2 import java.util.List;
3 class YouTubeChannel {
4     private List<String> subscribers = new ArrayList<>();
5     private String video;
6
7     // Method to add a new subscriber
8     public void addSubscriber(String subscriber) {
9         subscribers.add(subscriber);
10    }
11
12    // Method to upload a new video
13    public void uploadNewVideo(String video) {
14        this.video = video;
15        notifySubscribers(); // Notify all subscribers about the new video
16    }
17
18    // Notify all subscribers
19    public void notifySubscribers() {
20        for (String subscriber : subscribers) {
21            System.out.println(
22                "Notifying " + subscriber + " about new video: " + video);
23        }
24    }
25 }
```

```
class YouTubeSubscriber {  
    private String name;  
    public YouTubeSubscriber(String name) {  
        this.name = name;  
    }  
    public void subscribe(YouTubeChannel channel) {  
        channel.addSubscriber(name);  
    }  
    public void watchVideo(YouTubeChannel channel) {  
        System.out.println(name + " is watching the video: " + channel.video);  
    }  
}
```

What is the issue??

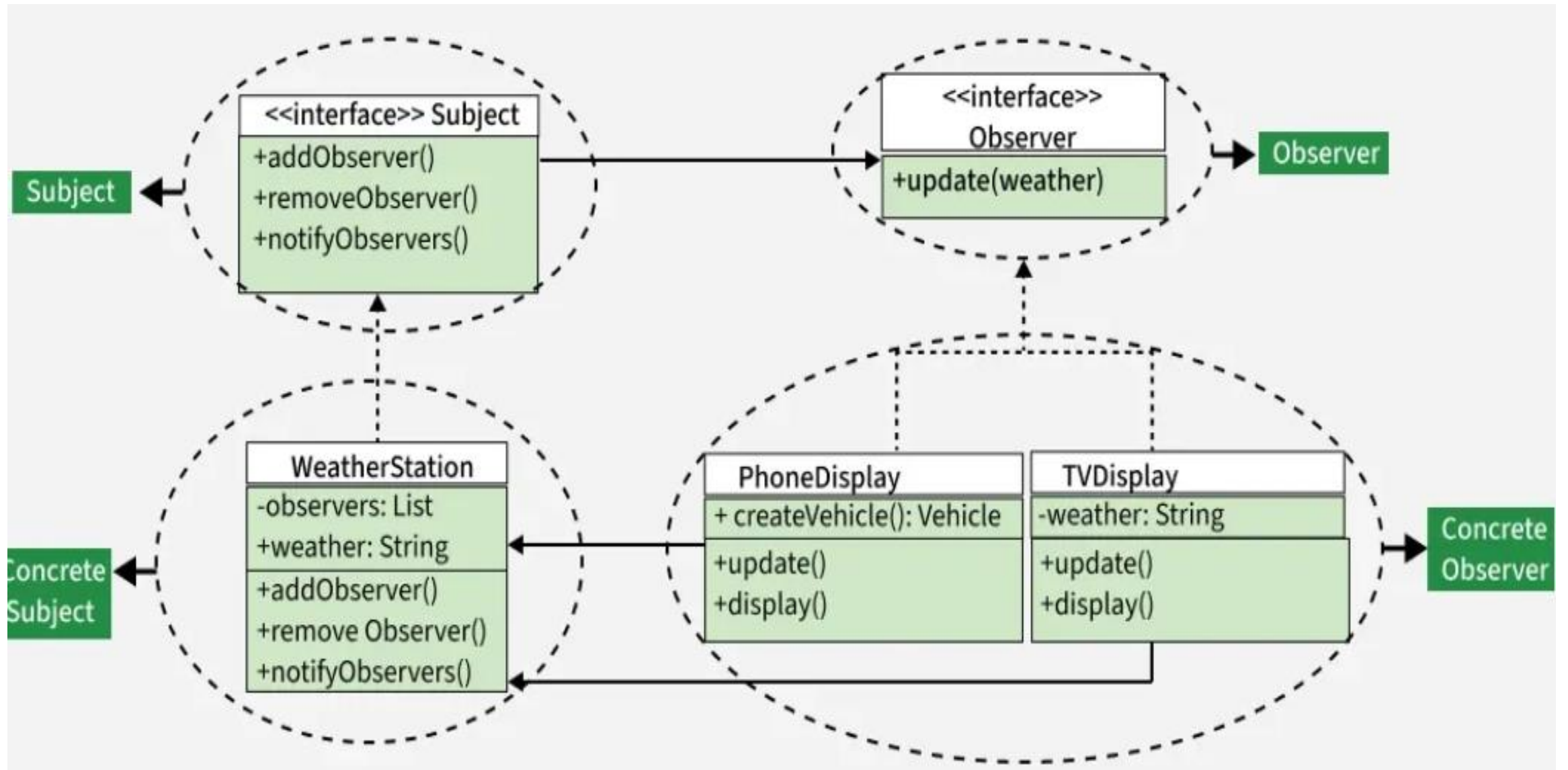
Solution

Observer Design Pattern - YouTube Channel Example



Hands-on

Consider a scenario where you have a weather monitoring system. Different parts of your application need to be updated when the weather conditions change.



Template Method pattern

"The Skeleton Framework."

Template Method Pattern

Problem??

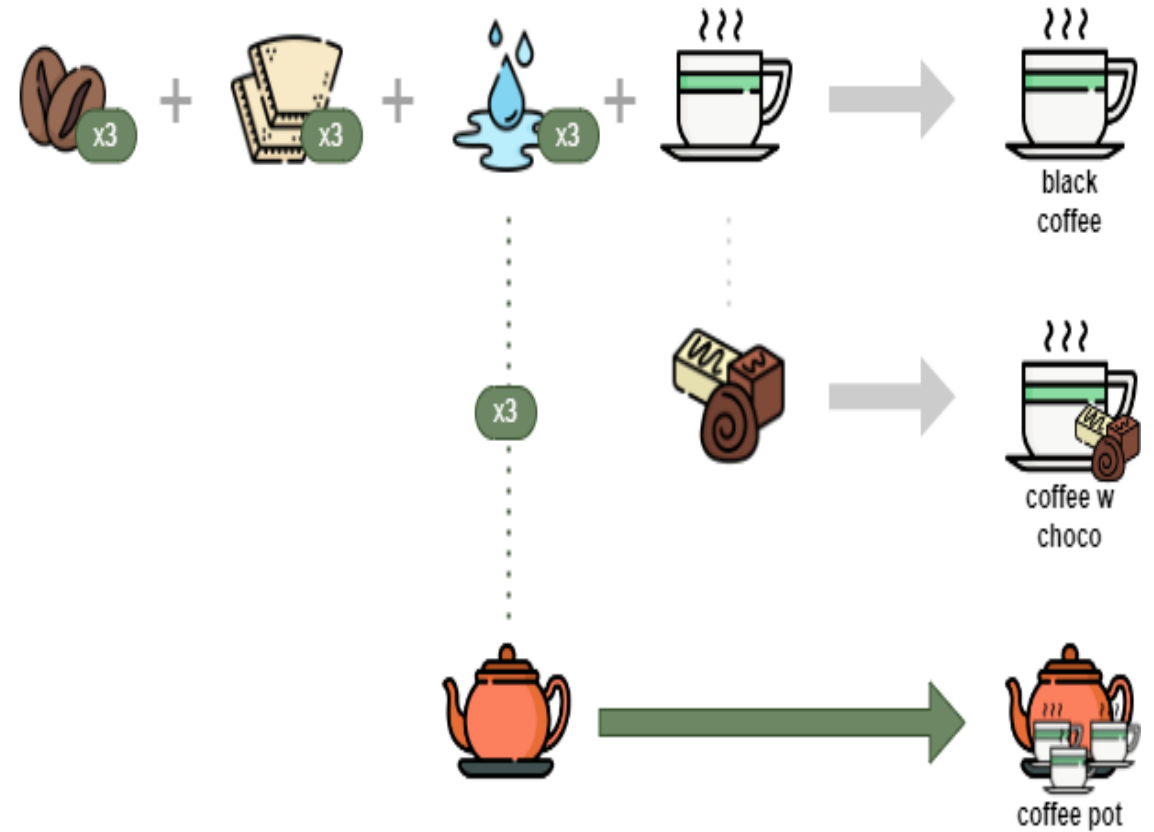
- Two different classes follow almost identical steps in a process, leading to code duplication.

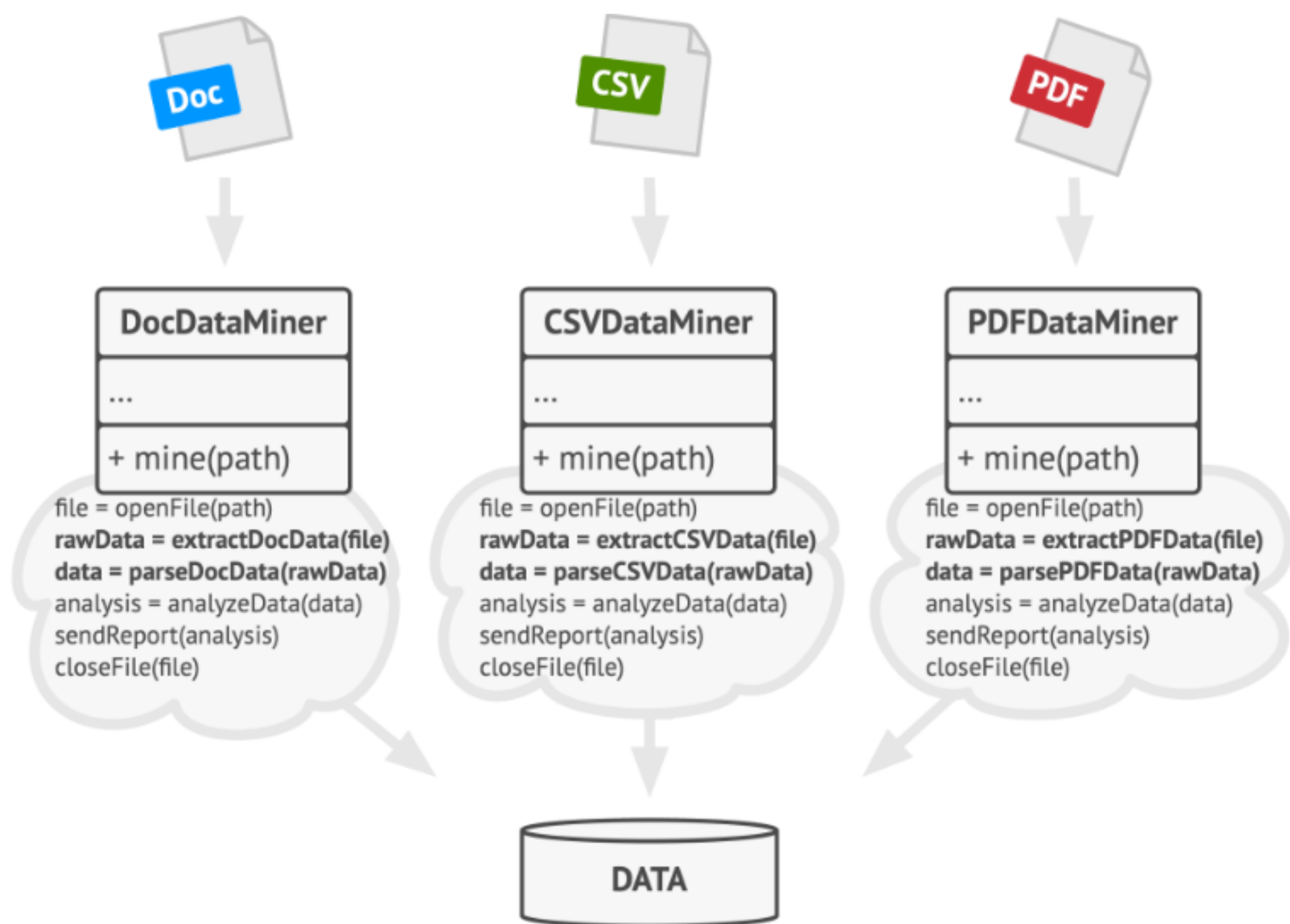
The Solution

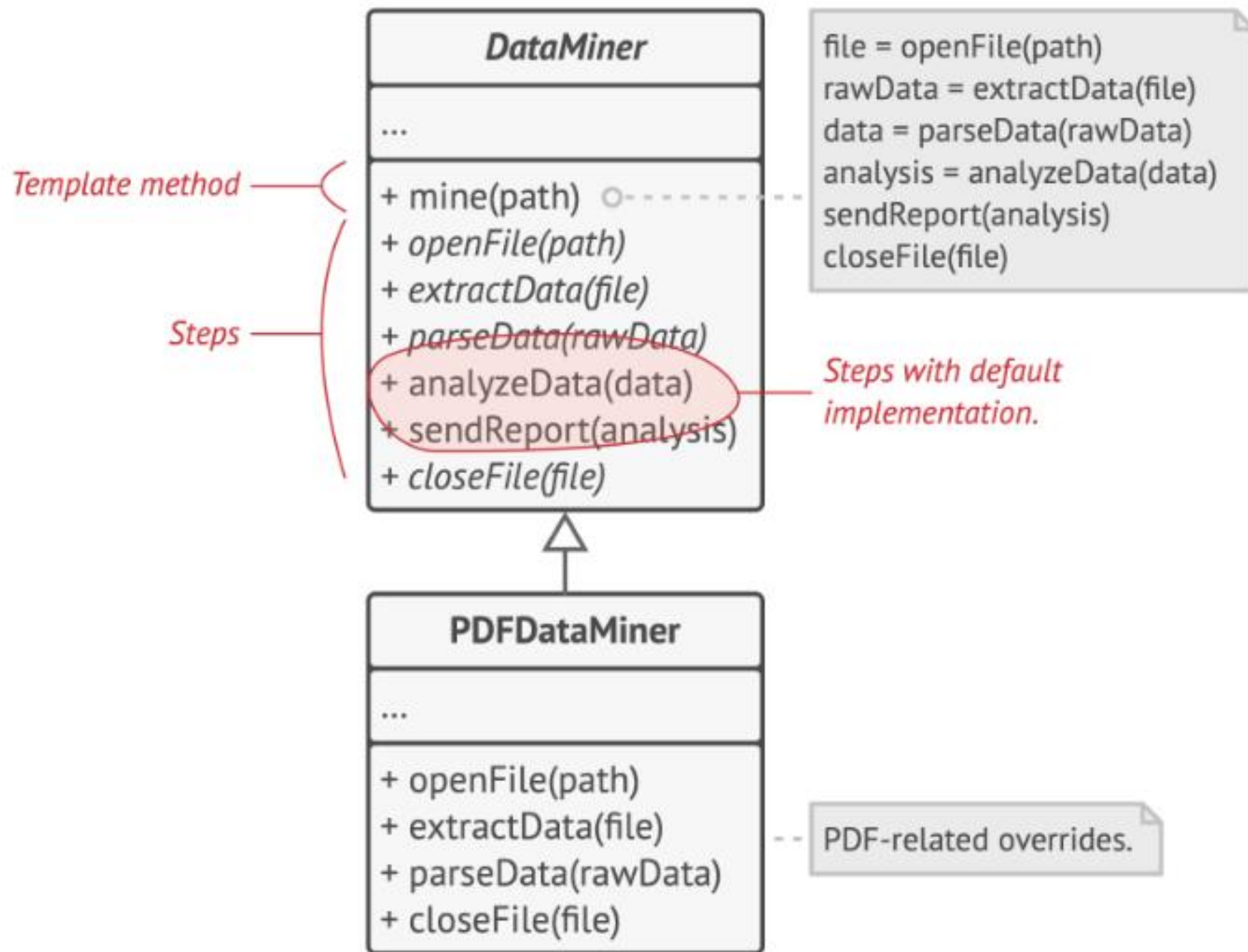
- Create a base class with a **final** method that calls a series of steps. Some steps are implemented in the base (the "Template"), while others are left abstract for subclasses to fill in.

Template Method Pattern

- Template Method Design Pattern is a behavioral pattern that defines the skeleton of an algorithm in a base method while allowing subclasses to override specific steps without altering its overall structure. It's like a recipe: the main steps remain fixed, but details can be customized for variation.







Components of Template Method Pattern

Abstract Class/Interface

- Defines the template method (algorithm skeleton) with some steps implemented and others left abstract or as hooks for customization.

Template Method

- Outlines the algorithm's fixed structure by calling steps in order, often marked *final* to prevent changes.

Abstract/Hook Methods:

- Placeholder methods in the abstract class that subclasses implement or optionally override.

Concrete Subclasses:

- Provide implementations for abstract methods, customizing specific steps while preserving the overall algorithm.

Click to enlarge

Abstract
Methods

Provide concrete implementations
for the abstract methods defined in
the superclass

Abstract Class

.....

+ templateMethod()
+ step1()
+ step2()
+ step3()
+ finalStep()

templateMethod()

step1()
step2()
step3()
finalStep()

ConcreteClassA

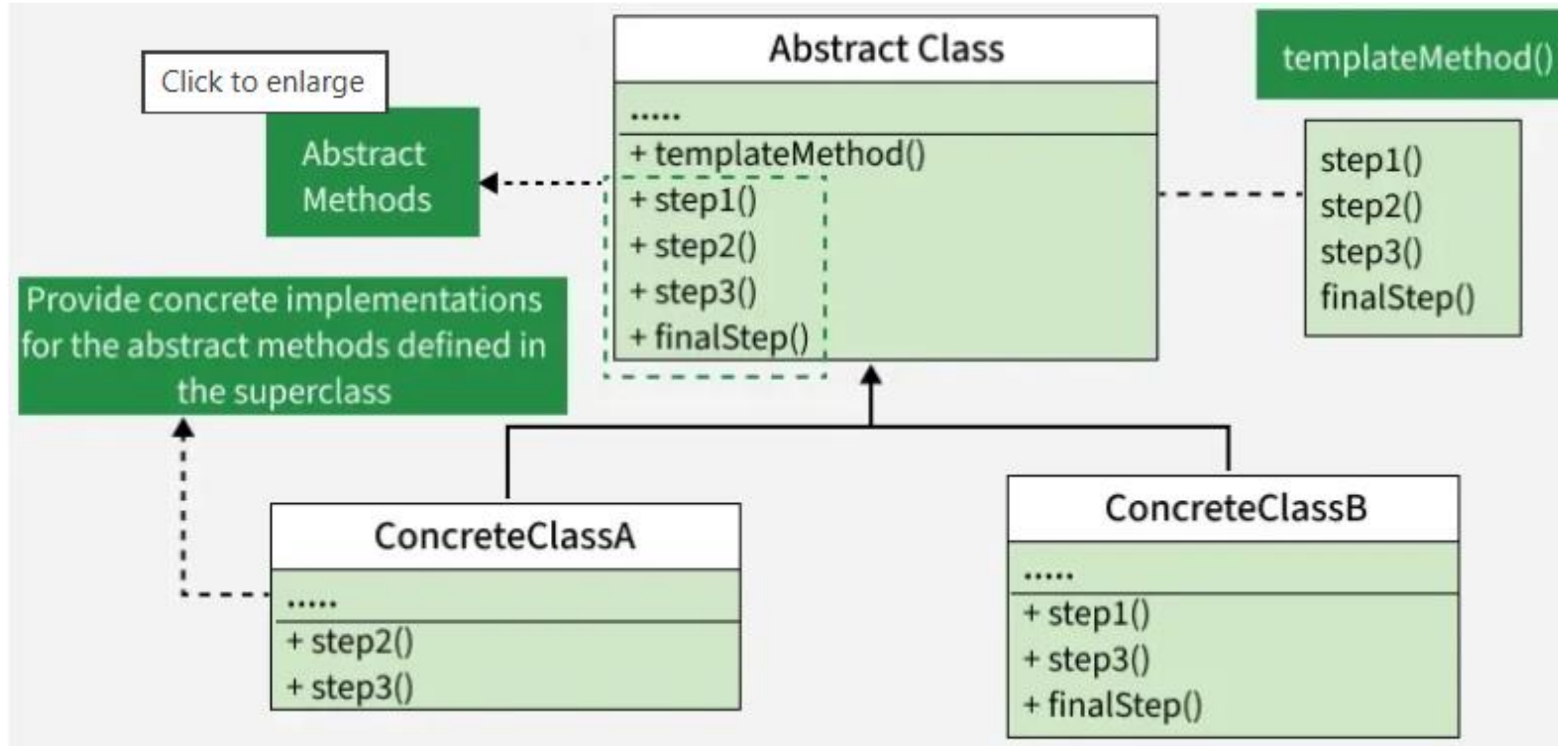
.....

+ step2()
+ step3()

ConcreteClassB

.....

+ step1()
+ step3()
+ finalStep()



Hands-on

```
/* Abstract class defining the template method */
abstract class BeverageMaker {
    // Template method defining the overall process
    void makeBeverage() {
        this.boilWater();
        this.brew();
        this.pourInCup();
        this.addCondiments();
    }

    // Abstract methods to be implemented by subclasses
    abstract void brew();
    abstract void addCondiments();

    // Common methods
    void boilWater() {
        System.out.println("Boiling water");
    }

    void pourInCup() {
        System.out.println("Pouring into cup");
    }
}
```

```
class TeaMaker extends BeverageMaker {
    // Implementing abstract methods
    @Override
    void brew() {
        System.out.println("Steeping the tea");
    }

    @Override
    void addCondiments() {
        System.out.println("Adding lemon");
    }
}
```

```
/* Concrete subclass for making coffee */  
class CoffeeMaker extends BeverageMaker {  
    // Implementing abstract methods  
    @Override  
    void brew() {  
        System.out.println("Dripping coffee through filter");  
    }  
  
    @Override  
    void addCondiments() {  
        System.out.println("Adding sugar and milk");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Making tea:");  
        BeverageMaker teaMaker = new TeaMaker();  
        teaMaker.makeBeverage();  
  
        System.out.println("\nMaking coffee:");  
        BeverageMaker coffeeMaker = new CoffeeMaker();  
        coffeeMaker.makeBeverage();  
    }  
}
```

Use of the Template Method Design Pattern

- **Common Algorithm with Variations:**
- **Code Reusability:**
- **Enforcing Structure:**
- **Reducing Duplication:**

Limitations of Using the Template Method Design Pattern

- **When Algorithms are Highly Variable:** Using the Template Method pattern might not be appropriate if the algorithms you're dealing with have a lot of differences in their steps and structure and little in common. This is because it could result in inappropriate abstraction or excessive complexity.
- **Tight Coupling Between Steps:** The Template Method pattern might not offer enough flexibility if there is a close coupling between the algorithm's parts, so modifications to one step require modifications to other steps.

Strategy pattern

"Pick a Tool for the Job."

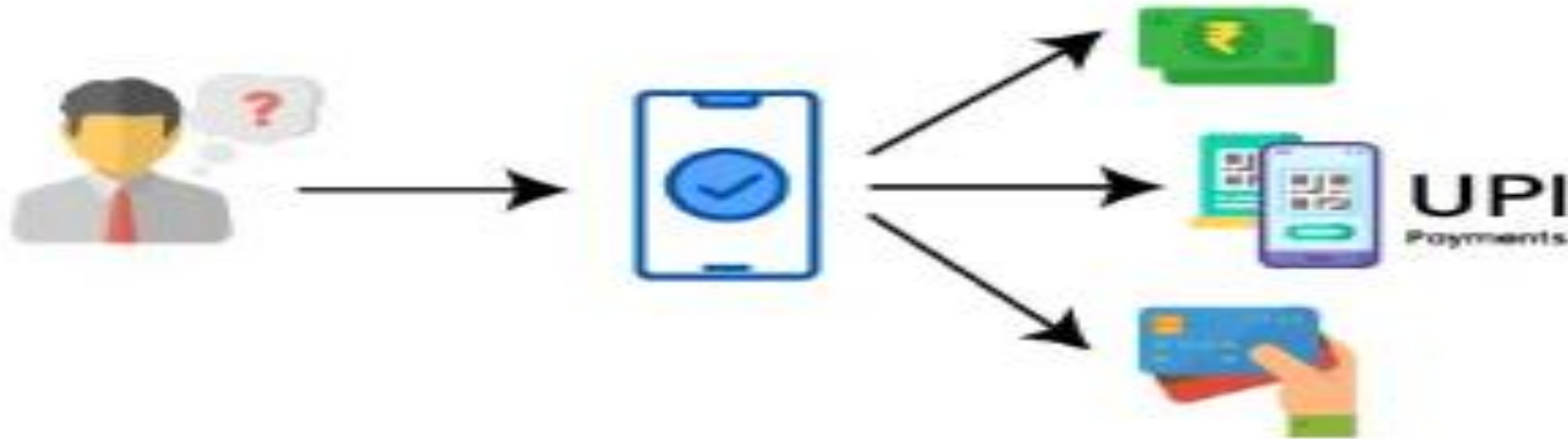
Strategy pattern

Problem??

- Hard-coded algorithms lead to rigid code and "Conditional Hell" (if/else or switch blocks).

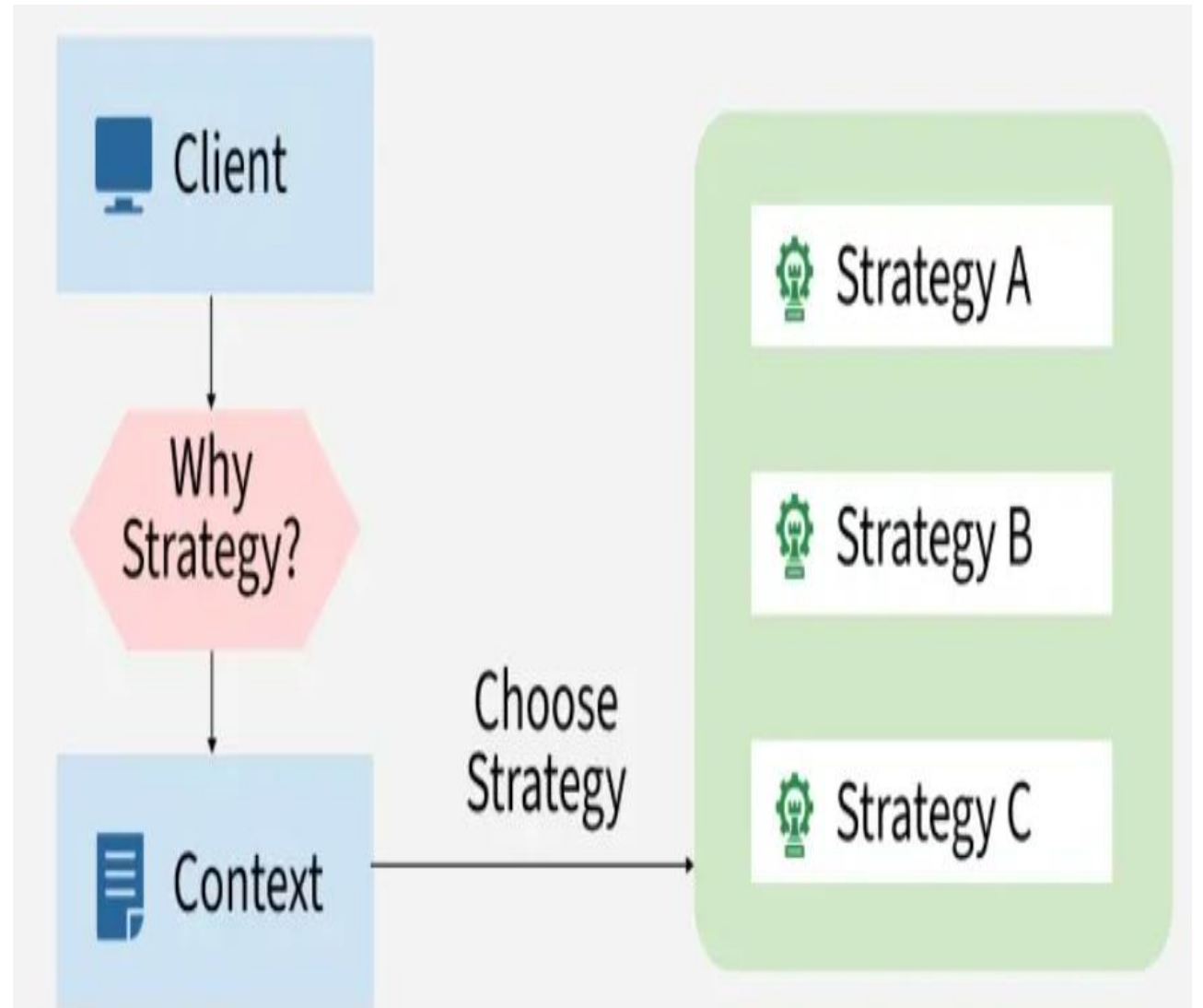
The Solution

- Encapsulate algorithms into separate classes that implement a common interface.



Strategy pattern

- **Strategy** is a behavioral design pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable.



Components of Strategy Pattern

Context

- assigns the task to a strategy object and contains a reference to it.
- intermediary between the client and the strategy
- offering an integrated approach for task execution without exposing every detail of the process.

Client

- creates an instance of the desired concrete strategy and passes it to the Context
- responsible for selecting and configuring the appropriate strategy and providing it to the Context.

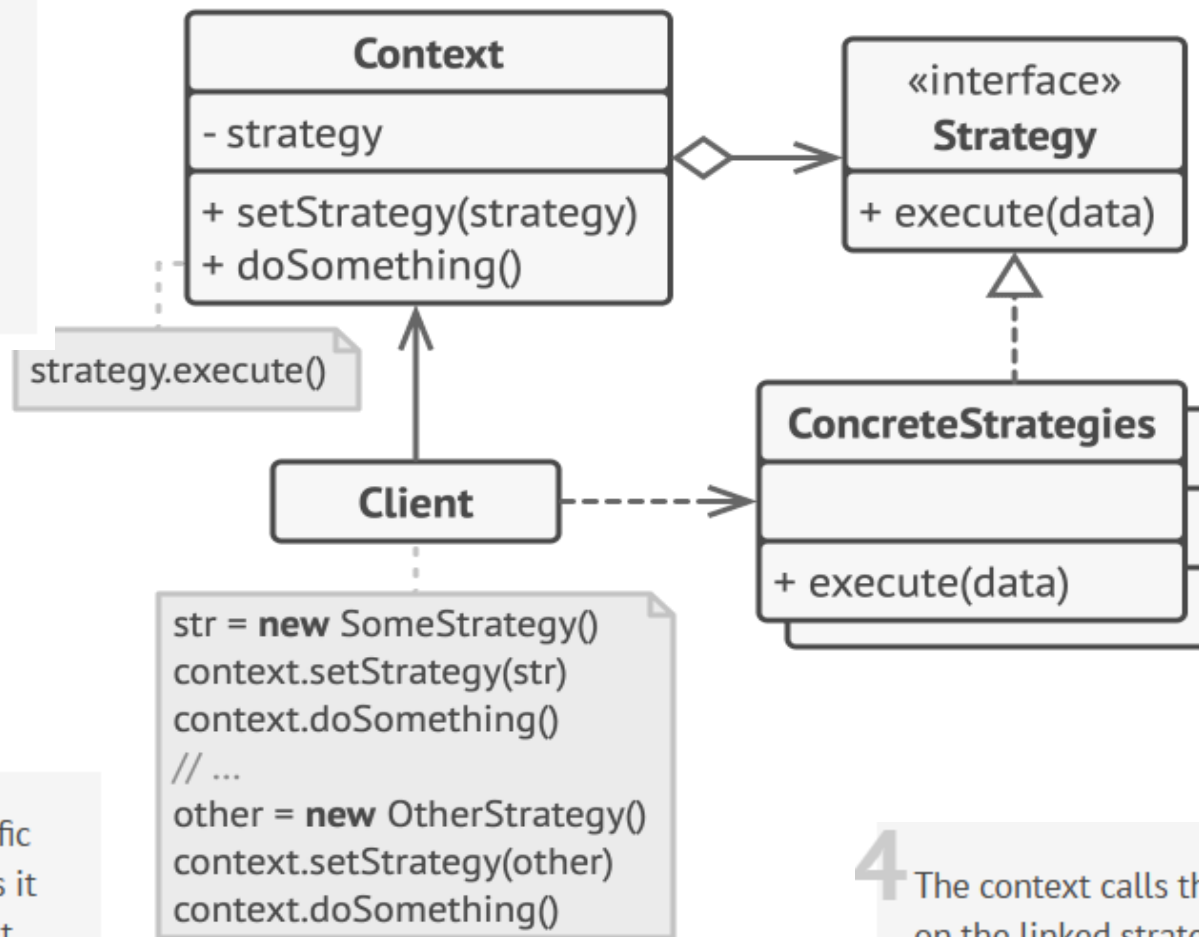
Strategy Interface

- specifies a set of methods that all concrete strategies must implement.
- guarantees that all strategies are interchangeable by the Context.
- Establishes a common interface that enables decoupling between the Context and the specific strategies.

Concrete Interface

- provides a specific algorithm or behavior for performing the task defined by the Strategy Interface.
- are interchangeable and can be selected and configured by the client based on the requirements of the task

1 The **Context** maintains a reference to one of the concrete strategies and communicates with this object only via the strategy interface.



2 The **Strategy** interface is common to all concrete strategies. It declares a method the context uses to execute a strategy.

3 **Concrete Strategies** implement different variations of an algorithm the context uses.

4 The context calls the execution method on the linked strategy object each time it needs to run the algorithm. The context doesn't know what type of strategy it works with or how the algorithm is executed.

5 The **Client** creates a specific strategy object and passes it to the context. The context exposes a setter which lets clients replace the strategy associated with the context at runtime.

Example 1

Calculator operations

```
// The strategy interface declares operations common to all
// supported versions of some algorithm. The context uses this
// interface to call the algorithm defined by the concrete
// strategies.
```

```
interface Strategy is
    method execute(a, b)
```

```
// Concrete strategies implement the algorithm while following
// the base strategy interface. The interface makes them
// interchangeable in the context.
```

```
class ConcreteStrategyAdd implements Strategy is
    method execute(a, b) is
        return a + b
```

```
class ConcreteStrategySubtract implements Strategy is
    method execute(a, b) is
        return a - b
```

```
class ConcreteStrategyMultiply implements Strategy is
    method execute(a, b) is
        return a * b
```

```
// The context defines the interface of interest to clients.  
class Context is  
    // The context maintains a reference to one of the strategy  
    // objects. The context doesn't know the concrete class of a  
    // strategy. It should work with all strategies via the  
    // strategy interface.  
    private strategy: Strategy  
  
    // Usually the context accepts a strategy through the  
    // constructor, and also provides a setter so that the  
    // strategy can be switched at runtime.  
    method setStrategy(strategy Strategy) is  
        this.strategy = strategy  
  
    // The context delegates some work to the strategy object  
    // instead of implementing multiple versions of the  
    // algorithm on its own.  
    method executeStrategy(int a, int b) is  
        return strategy.execute(a, b)
```

```
// The client code picks a concrete strategy and passes it to
// the context. The client should be aware of the differences
// between strategies in order to make the right choice.
class ExampleApplication is
    method main() is
        Create context object.

        Read first number.
        Read last number.
        Read the desired action from user input.

        if (action == addition) then
            context.setStrategy(new ConcreteStrategyAdd())

        if (action == subtraction) then
            context.setStrategy(new ConcreteStrategySubtract())

        if (action == multiplication) then
            context.setStrategy(new ConcreteStrategyMultiply())

        result = context.executeStrategy(First number, Second number)

        Print result.
```

Example 1

Payment process

Remember this??

```
package withoutstrategy;

public class PaymentProcessor {
    private PaymentType paymentType;

    public void processPayment(double amount) {
        if (paymentType == PaymentType.CREDIT_CARD) {
            System.out.println("Processing credit card payment of amount " + amount);
        } else if (paymentType == PaymentType.DEBIT_CARD) {
            System.out.println("Processing debit card payment of amount " + amount);
        } else if (paymentType == PaymentType.PAYPAL) {
            System.out.println("Processing PayPal payment of amount " + amount);
        } else {
            throw new IllegalArgumentException("Invalid payment type");
        }
    }

    public void setPaymentType(PaymentType paymentType) {
        this.paymentType = paymentType;
    }
}
```

```
package withstrategy;

public class DebitCardPaymentStrategy implements PaymentStrategy {
    public void processPayment(double amount) {
        System.out.println("Processing debit card payment of amount " + amount);
    }
}
```

```
package withstrategy;

public class PaypalPaymentStrategy implements PaymentStrategy {
    public void processPayment(double amount) {
        System.out.println("Processing PayPal payment of amount " + amount);
    }
}
```

```
package withstrategy;

public class CreditCardPaymentStrategy implements PaymentStrategy {
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment of amount " + amount);
    }
}
```

```
package withstrategy;

public class PaymentProcessor {
    private PaymentStrategy paymentStrategy;

    public PaymentProcessor(PaymentStrategy paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }

    public void processPayment(double amount) {
        paymentStrategy.processPayment(amount);
    }
}
```

```
package withstrategy;

public interface PaymentStrategy {
    void processPayment(double amount);
}
```

Use of the Strategy Design Pattern

- Multiple algorithm variants needed
- Eliminating conditional statements
- Runtime algorithm selection